MSDI Calculator

Lookup Table Development, Generation, and QA/QC Methodology

Technical Methodology Memorandum

Prepared for internal model documentation and technical review

Prepared by: Nathan Sparrow

Prepared date: July 8, 2026

Model scope: Manure Sub-Surface Drip Irrigation (mSDI-e) economic screening calculator

Document status: Methodology documentation for lookup-table source control, generation, validation, and change management based on current workbook and TypeScript implementation.

Purpose and Scope

This technical memorandum documents the lookup-table development, generation, and quality-control methodology used by the MSDI Calculator. Lookup tables provide the source coefficients, selection values, labels, county/weather records, cost assumptions, crop and soil coefficients, manure coefficients, utility assumptions, and output text used by the application. The methodology describes the role of the Excel workbook as the source of truth, the manual update and review workflow, the lookup-generation script, the generated JSON architecture, type/key access patterns, change management, and validation expectations.

The methodology is intended for internal model documentation, technical review, and future consolidation into the broader MSDI Calculator methodology manual. It does not independently justify every coefficient value; coefficient-specific source development is documented in separate methodology memoranda. This document focuses on how lookup data are maintained and incorporated into the application in a reproducible and reviewable manner.

Terminology and Units

Term	Definition in this methodology	Primary artifact or unit
Source workbook	The manually maintained Excel workbook that stores approved lookup tables before conversion to application-readable JSON.	msdi_lookup_tables.xlsx
Lookup sheet	A worksheet in the source workbook representing one logical lookup table, typically named with an _LU suffix.	Excel worksheet
Lookup JSON	Generated JSON file created from a workbook sheet and imported by the application.	lookup-*.json
Lookup key	Stable string identifier used by application logic to retrieve a row or value from a lookup table.	string
Lookup record	A row-level object in a generated lookup JSON file containing normalized field names, label, key, and value fields where applicable.	JSON object
Source of truth	The controlled workbook and change log used as the primary editable record for lookup updates.	Excel + CHANGELOG.md
QA/QC gate	Review step completed before regenerated JSON files are accepted into the application.	manual + automated checks

Source Workbook as the Source of Truth

The MSDI Calculator uses an Excel workbook as the editable source of truth for lookup-table content. The uploaded workbook contains lookup sheets including States_LU, County_LU, Weather_LU, Cost_LU, Depreciation_LU, Coefficients_LU, Manure_Filter_LU, Manure_Nutrient_LU, Animal_Class_LU, Manure_App_LU, Manure_LU, Salts_LU, Soils_LU, Rodents_LU, Weeds_LU, Tillage_LU, Crops_LU, Rotation_LU, Utilities_LU, Water_Right_LU, Source_LU, Verbage_LU, and Irrigation_LU. These sheets are translated into JSON and imported by the app through the lookup index.

The workbook-first workflow is intentional. Technical assumptions can be reviewed in tabular form, coefficient owners can edit lookup values without changing application code, and source comments or supporting columns can be retained adjacent to the coefficients. Once a workbook update is approved, generated outputs are rebuilt using the lookup generation script rather than edited by hand.

Workbook element	Role in methodology	Implementation implication
Sheet name	Defines the generated lookup table name after removal of the _LU suffix and conversion to kebab-case.	Example: Cost_LU -> lookup-cost.json.
Header row	Defines generated object field names after conversion to snake_case.	Column names must be stable because TypeScript accessors rely on generated field keys.
Key column	Optional stable key field used when present.	If present, the generator uses the provided key rather than deriving one from the first column.
First column	Original label field and fallback key source.	The generator always stores the first column value as label.
Blank rows or blank first column	Ignored during generation.	Prevents empty workbook rows from becoming lookup records.

Manual Lookup Updates and Rationale for Not Using Live APIs

Lookup updates are incorporated manually through the workbook and change-management process rather than through live API calls during calculation. This is a core reproducibility decision. The MSDI Calculator is a screening-level model whose outputs should be stable across runs unless a documented dataset or coefficient update is intentionally incorporated.

Live APIs are not used as the runtime source for lookup values for five main reasons. First, version control requires a known set of values that can be committed, reviewed, and traced. Second, QA/QC must occur before new values affect model outputs. Third, reproducibility requires the ability to rerun a prior model version using the same lookup values. Fourth, datasets must be frozen for documentation, review, and methodology alignment. Fifth, several model variables have no reasonable API source because they are engineering assumptions, vendor-quote values, internal screening coefficients, or controlled text strings rather than public data products.

Reason APIs are not used	Methodological rationale
Version control	Lookup values must be reviewed as explicit workbook and generated-output changes rather than changing dynamically outside the repository.
QA/QC before incorporation	New coefficient values should be checked for units, ranges, keys, and downstream output effects before they affect users.
Reproducibility	A prior result should be explainable using the lookup version in effect when the result was generated.
Dataset freezing	Weather, cost, manure, crop, soil, and output-text assumptions may need to remain fixed for a methodology release.
No reasonable API source	Many values are custom screening assumptions, vendor-derived costs, generated coefficients, or legal/output text that cannot be fetched from a public API.

Lookup Generation Workflow

Lookup JSON files are generated from the workbook using the project package script pnpm run generate:lookups. The uploaded generation script reads src/data/raw/msdi_lookup_tables.xlsx, iterates through each worksheet, converts header names to snake_case, builds a keyed object for each sheet, and writes one formatted JSON file per sheet to src/data/lookups.

The generator normalizes yes/no string values to booleans when used in value fields. If a workbook row does not contain a value field, the generator creates one from the row key. Each generated record always includes label and key, which preserves both the original user-facing label and the stable lookup identifier.

1. Update the appropriate worksheet in msdi_lookup_tables.xlsx.
2. Confirm headers, key fields, numeric units, and source notes are complete.
3. Run pnpm run generate:lookups from the project root.
4. Review the regenerated lookup-*.json files in src/data/lookups.
5. Run representative model scenarios or targeted tests affected by the changed lookup table.
6. Document the update in CHANGELOG.md and reference the supporting methodology or data source.

Generation step	Implemented behavior
Read workbook	XLSX.readFile loads src/data/raw/msdi_lookup_tables.xlsx.
Normalize headers	Headers are converted to lowercase snake_case with non-word characters removed.
Select key	Existing key column is used if present; otherwise a snake_case key is generated from the first column.
Create record	Non-empty cells are assigned to the generated field names.
Preserve label/key	label stores the original first-column value; key stores the lookup identifier.
Normalize value	yes/no strings are converted to boolean values where applicable.
Write JSON	Each sheet is written as lookup-{sheet-name}.json after removing _LU and converting to kebab-case.

Generated JSON Lookup Files and Import Architecture

Generated lookup JSON files are imported into the application through the lookup index module. The index imports each lookup JSON file and exposes a lookupTables object keyed by table name. It also pre-groups counties by state FIPS for efficient state/county selection in the user interface. The index currently includes lookup_animal_class, lookup_coefficients, lookup_cost, lookup_counties, lookup_crops, lookup_depreciation, lookup_elevation, lookup_irrigation, lookup_manure, lookup_manure_app, lookup_manure_filter, lookup_manure_nutrient, lookup_right_of_way, lookup_rodents, lookup_rotation, lookup_salts, lookup_soils, lookup_source, lookup_states, lookup_tillage, lookup_utilities, lookup_verbage, lookup_water_right, lookup_weather, and lookup_weeds.

Architecture element	Function
src/data/raw/msdi_lookup_tables.xlsx	Editable source workbook for approved lookup values.
scripts/generate-lookups.ts or equivalent package script	Builds generated JSON files from the workbook.
src/data/lookups/lookup-*.json	Application-readable lookup data generated from workbook sheets.
src/data/lookups/index.ts	Imports generated lookup files and exports lookupTables.
src/lib/lookup-utils.ts	Provides getLookupJson, getLookupRecord, and getLookupValue accessors.
src/lib/coefficients.ts	Provides typed coefficient accessors backed by lookup_coefficients.
src/types/lookup-keys	Defines stable key constants and typed key names used by calculation modules.

Lookup Key and Type Architecture

The lookup architecture separates human-readable labels from stable machine keys. User-facing text can be revised where appropriate, but row keys and column field names should be treated as implementation interfaces. Changing a key without updating all TypeScript references can break model execution or silently change which lookup record is retrieved.

Lookup utility functions enforce table and record existence at runtime. getLookupJson throws an error if a lookup table is missing, and getLookupRecord throws an error if an entry key is missing. getLookupValue returns a field value from a validated record. Coefficient access is further typed through getCoeff and coefficient-specific accessors, which convert lookup values to numbers and throw a TypeError if a coefficient is not numeric.

Pattern	Purpose	QA/QC implication
Table key constants	Avoid hard-coded table strings scattered through the codebase.	Confirm new tables are added to the index and type definitions.
Row key constants	Provide stable identifiers for lookup rows used by calculations.	Avoid changing keys unless all references are updated.
Column constants	Provide stable field names for generated JSON values.	Header changes in Excel must be reflected in type constants and code references.
Typed coefficient accessors	Centralize unit conversion and numeric validation for coefficients.	Non-numeric coefficient values are detected when accessed.
Runtime missing-record errors	Prevent silent fallback when required lookup records are unavailable.	Representative test scenarios should exercise updated lookup rows.

Change Management and Changelog Requirements

Lookup changes should be documented in CHANGELOG.md at the same time the source workbook and generated outputs are updated. The uploaded change log shows this practice for weather lookup updates, RCP climate integration, seasonal effective rainfall coefficients, manure management coefficients, manure application equipment performance coefficients, output labels, disclaimer updates, salvage handling, and CSV/PDF/HTML result alignment.

A change-log entry should describe the lookup table or code area updated, the reason for the change, source data or methodology basis, generated outputs, expected model impact, and validation completed. When a lookup change affects downstream calculations, the change entry should identify affected modules and whether output values changed in representative test scenarios.

Change-log field	Recommended content
Files updated	Workbook sheet, generated lookup JSON files, TypeScript files, and methodology documents affected.
Summary	Plain-language description of the change.
Details	Specific fields, coefficients, rows, or generated structures changed.
Methodology or source	Reference to internal methodology, vendor quote, data product, or literature source.
Impact	Expected model areas affected, such as OPEX, crop water demand, capital cost, weather scenarios, or reporting.
Validation	Script execution, output comparison, representative scenario checks, and key/range validation completed.

QA/QC and Validation Workflow

Lookup QA/QC occurs before, during, and after generation. Workbook QA focuses on source values, units, key stability, blank rows, and source notes. Generation QA verifies that the package script completes successfully and produces expected JSON files. Application QA verifies that updated lookup values are retrieved correctly and do not create unexpected output changes.

QA/QC check	Purpose
Confirm workbook is the edited source	Avoid direct edits to generated JSON without corresponding workbook changes.
Validate sheet names and headers	Prevent unintended file names or field names after snake_case/kebab-case conversion.
Check key uniqueness	Prevent duplicate keys from overwriting records in generated JSON objects.
Check units and numeric formats	Avoid mixing inches, gallons, tons, dollars, fractions, percentages, or text-formatted numbers.
Review generated JSON diffs	Confirm only intended lookup records changed.
Run representative scenarios	Verify affected outputs remain plausible and aligned with methodology.
Check missing lookup errors	Confirm TypeScript accessors can find required tables, records, and values.
Update changelog and methodology links	Preserve traceability between source values, code, and documentation.

Assumptions and Limitations

- Lookup values are screening-level model assumptions unless documented otherwise in a coefficient-specific methodology.
- The Excel workbook is the editable source of truth; generated JSON files are derived implementation artifacts.
- The generator does not independently validate scientific correctness, units, plausible ranges, or source quality.
- Header changes can affect generated field names and should be treated as code-interface changes.
- Key changes can break downstream lookup access unless matching TypeScript constants and references are updated.
- Manual update workflows require disciplined review but provide better reproducibility than live runtime data pulls.
- Frozen lookup datasets may become stale over time and should be periodically reviewed through a documented revision cycle.

Processing Workflow Summary

1. Identify the lookup table requiring update and confirm whether a coefficient-specific methodology or source document applies.
2. Update the appropriate worksheet in msdi_lookup_tables.xlsx using stable labels, keys, units, and source notes.
3. Review workbook fields for blank rows, duplicate keys, inconsistent units, and unintended header changes.
4. Run pnpm run generate:lookups from the project root.
5. Confirm generated lookup JSON files are created in src/data/lookups with expected file names and changed values.
6. Verify lookup index and type/key definitions remain aligned with the generated files.
7. Run representative app scenarios or targeted tests for affected modules.
8. Compare HTML, PDF, CSV, and model outputs when lookup changes affect reporting or calculations.
9. Document the update, source basis, impact, and validation in CHANGELOG.md.
10. Commit the workbook, generated lookup files, changelog, and any required code or methodology updates together.

Implementation Traceability

Methodology area	Primary implementation file or artifact	Traceability note
Source workbook	src/data/raw/msdi_lookup_tables.xlsx	Editable source of truth for lookup values.
Lookup generation	generate-lookups.ts; pnpm run generate:lookups	Reads workbook sheets and writes lookup-*.json files.
Generated lookup imports	src/data/lookups/index.ts	Imports generated JSON files and exposes lookupTables.
Lookup retrieval	src/lib/lookup-utils.ts	Provides table, record, and field accessors with missing-table and missing-record errors.
Coefficient access	src/lib/coefficients.ts	Provides numeric coefficient accessors backed by lookup_coefficients.
Change management	CHANGELOG.md	Documents lookup, logic, output, and methodology changes.
Workbook sheet list	msdi_lookup_tables.xlsx worksheets	Defines lookup domains including weather, costs, crops, soils, manure, utilities, rodents, weeds, tillage, and reporting text.

References and Source Links

MSDI Calculator source workbook. msdi_lookup_tables.xlsx. Internal project lookup workbook, June 2026.

MSDI Calculator lookup generation script. generate-lookups.ts. Internal TypeScript implementation, June 2026.

MSDI Calculator lookup utilities. src/lib/lookup-utils.ts and src/lib/coefficients.ts. Internal TypeScript implementation, June 2026.

MSDI Calculator lookup index. src/data/lookups/index.ts. Internal TypeScript implementation, June 2026.

MSDI Calculator Change Log. CHANGELOG.md. Internal project change management record, 2026.

Related internal methodology memoranda: Capital Cost Methodology; Crop and Soil Coefficients Methodology; Crop Water Demand and Irrigation Requirement Methodology; Manure Nutrient and Application Mass Balance Methodology; Operational Cost Methodology; Weather Data Methodology.